

INTRODUCCIÓN AL UML

1	<i>Representación de problemas del mundo real: la abstracción</i>	1
2	<i>El paradigma de la orientación a objetos</i>	2
3	<i>El Lenguaje Unificado de Modelado</i>	4
3.1	Concepto de UML	4
3.2	Diagramas en UML	5
3.3	Utilización de UML en la metodología RUP	6
4	<i>Bibliografía</i>	7

1 Representación de problemas del mundo real: la abstracción

En el mundo real se presentan un gran número de diferentes problemas a solucionar, problemas de la naturaleza, problemas sociales, económicos, de organización, etc.

Desde el principio, el hombre ha intentado explicar las causas del comportamiento del entorno en el que se encuentra. Para la realización de esta actividad el ser humano se ha basado en su capacidad de abstracción, potente herramienta mediante la que pueden ser estudiados los problemas del mundo real gracias a la simplificación que permite introducir en el proceso de representación de los mismos.

Cualquier problema que se desee representar puede ser visto como un sistema en el que intervienen una serie de elementos del mismo que mantienen un número de interdependencias entre ellos. Además, existirán otros problemas que afectarán a éste, o que se verán afectados por él.

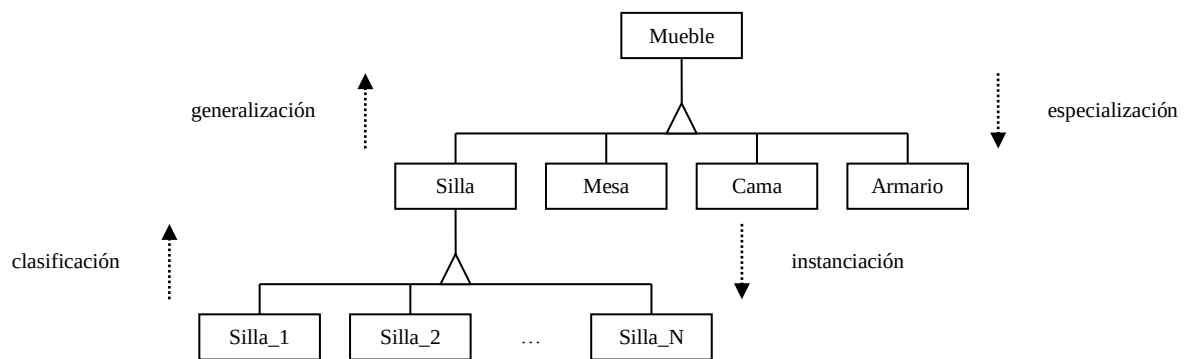
Mediante la abstracción una serie de objetos se caracterizan en un nuevo tipo de objetos mediante una función de pertenencia. Al nuevo tipo de objetos se le denomina clase, y todos los objetos caracterizados en la clase tienen propiedades y comportamientos comunes que quedan recogidos en la clase. Así pues, la abstracción permite ocultar los detalles, simplificando la descripción de un problema mediante la agrupación de los objetos con características comunes que intervienen en el mismo.

La abstracción es utilizada cotidianamente por el ser humano. Por ejemplo, el término “vehículo” es una clase mediante la que representamos a aquellos objetos que tienen el comportamiento de desplazarse sobre ruedas. Los elementos de esta clase pueden ser: automóviles, camiones, autobuses, etc. Todos ellos son vehículos, y a su vez tienen un conjunto de características particulares que los diferencian del resto de los vehículos (por ejemplo, el número de plazas). Por otra parte, el término “automóvil” es otra clase que agrupa a los distintos tipos de automóviles (biplazas, berlinas, monovolúmenes, etc)

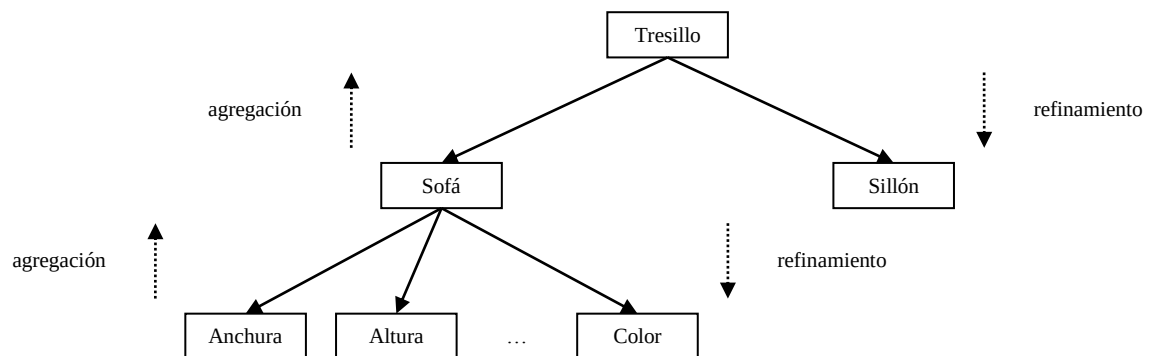
En la definición de datos la abstracción es utilizada de varias formas:

- La generalización (relación “ES UN”): capacidad de considerar un conjunto de objetos como una nueva clase más general. Las clases pueden ser a su vez generalizaciones de otras clases. La generalización de objetos simples se llama clasificación.

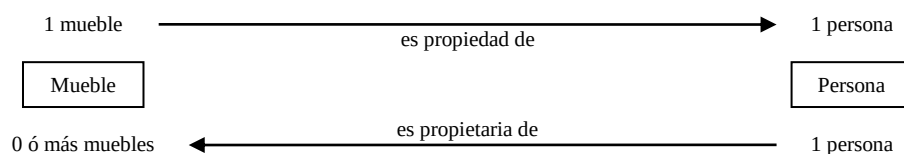
Se denomina especialización e instanciación a los procesos inversos a la generalización y clasificación, respectivamente.



- La agregación (relación “ES PARTE DE”): capacidad de considerar una clase basándose en los elementos que la constituyen. Más aún, en una clase podemos encontrar la agregación de un conjunto de propiedades que la caracterizan. El proceso inverso a la agregación es el refinamiento, mediante el cual se pueden especificar aquellas clases más simples o aquellas propiedades que caracterizan a una clase.



- Además de las relaciones “ES UN” y “ES PARTE DE” consideradas entre las clases que intervienen en un problema, en la mayor parte de los problemas existen otro tipo de relaciones o dependencias entre los elementos del mismo. Por ejemplo, consideremos que se desean representar los muebles que las personas tienen en su casa; de forma simplificada se puede deducir que en el problema están presentes dos clases: la clase “Mueble” y la clase “Persona”; entre ambas existe una dependencia o relación, la que identifica qué persona es propietaria de qué mueble. Además, vemos que una persona podrá poseer varios muebles, pero consideraremos que cada mueble sólo es propiedad de una persona.



2 El paradigma de la orientación a objetos

La programación orientada a objetos es un paradigma surgido en los años 70, que utiliza objetos como elementos fundamentales en la construcción de la solución. Con este paradigma el desarrollo de software se centra en simular de la forma más certera posible los elementos de la realidad asociados al problema que hay que resolver.

Un objeto es una representación de algún hecho o ente del mundo real. Todas las propiedades y métodos comunes a los objetos se encapsulan o agrupan en clases; cada objeto es una instancia o ejemplar de una

clase. Las aplicaciones orientadas a objetos están formadas por un conjunto de objetos que interactúan enviándose mensajes para producir resultados.

Un objeto está formado por:

- Un conjunto de atributos, que son los datos que le caracterizan.
- Un conjunto de operaciones que definen su comportamiento. Las operaciones asociadas a un objeto actúan sobre sus atributos para modificar su estado. Cuando se indica a un objeto que ejecute una operación determinada se dice que se le pasa un mensaje.

La orientación a objetos se basa en los siguientes principios:

1. Cualquier problema se puede modelar como un conjunto de objetos.

Ejemplos de objetos son los siguientes:

Lassie

Propiedades:	raza = "Collie de pelo largo", peso = 23 kg, altura de cruz = 53 cm, sexo = hembra, ...
Comportamientos:	ladrar, comer, dormir, correr,...

Avión "Teresa de Ávila"

Propiedades:	propietario = "Iberia", Marca = "Airbus", Modelo = "A-340/300", número máximo de pasajeros = 295, número actual de pasajeros = 187, está el motor arrancado = sí, está en tierra = no,...
Comportamientos:	arrancar motores, parar motores, despegar, aterrizar, girar,...

2. Todo objeto es de algún tipo, es decir, debe ser creado a partir de una clase. El conjunto de propiedades y comportamientos esperados de un objeto es lo que determina a qué tipo o clase pertenece.

La clase es el molde con el que se construyen los objetos. Es la "fábrica" de los objetos. A los objetos que se crean a partir de una clase se les llama instancias de la clase. En el ejemplo anterior: el objeto "Lassie" es una instancia de la clase Perro.

Es necesario distinguir claramente ambos conceptos:

- Una Clase: Sólo es el molde de creación de los objetos. Por tanto, es un elemento estático, no evoluciona en el tiempo. A partir de una clase se pueden crear muchos objetos (instancias).
 - Un Objeto: Tiene valores concretos en las propiedades, que determinan su estado. Es un elemento dinámico; es decir, su estado evoluciona durante la ejecución del programa. Un objeto sólo puede ser creado a partir de una clase.
3. Los objetos se comunican mediante mensajes. Los mensajes son las peticiones que un objeto realiza a otro para que éste último realice alguno de sus comportamientos.
 4. Una aplicación orientada a objetos es una comunidad de objetos que nacen (se crean a medida que se necesitan), interactúan (se mandan mensajes) y mueren (cuando ya no se necesitan se destruyen y se libera la memoria).

A continuación se describen las características más importantes que contempla la "orientación a objetos":

- Abstracción

Denota las características esenciales de un objeto, incluyendo sus comportamientos; permite seleccionar las características relevantes e identificar comportamientos comunes para definir nuevas clases. La abstracción es clave en el proceso de análisis y diseño orientado a objetos, ya que mediante ella podemos llegar a armar un conjunto de clases que permitan modelar la realidad o el problema que se quiere atacar.

- Encapsulamiento

Significa reunir todos los elementos que pueden considerarse pertenecientes a un mismo objeto. Esto permite aumentar la cohesión de los componentes del sistema. Algunos autores confunden este concepto con el principio de ocultación, principalmente porque se suelen emplear conjuntamente.

- Principio de ocultación

Cada clase expone una interfaz a otros objetos que especifica cómo pueden interactuar con los objetos de la clase. Esto asegura que otros objetos no puedan cambiar el estado interno de un objeto de manera inesperada; así se protegen las propiedades del objeto contra su modificación por quien no tenga derecho a acceder a ellas; solamente los propios métodos del objeto pueden acceder a su estado.

- Modularidad

Se denomina modularidad a la propiedad que permite subdividir una aplicación en partes más pequeñas (llamadas módulos), cada una de las cuales debe ser tan independiente como sea posible de la aplicación en sí y de las restantes partes. Estos módulos se pueden compilar por separado, pero tienen conexiones con otros módulos. En orientación a objetos las clases se pueden considerar los módulos más básicos del sistema.

- Polimorfismo

Comportamientos diferentes, asociados a objetos distintos, pueden compartir el mismo nombre; al llamarlos por ese nombre se utilizará el comportamiento correspondiente al objeto que se esté usando. O, dicho de otro modo, las referencias y las colecciones de objetos pueden contener objetos de diferentes tipos, y la invocación de un comportamiento en una referencia producirá el comportamiento correcto para el tipo real del objeto referenciado.

- Herencia

Relación que se establece entre clases en las que unas utilizan las propiedades y comportamientos de otras formando una jerarquía de especialización. Los objetos heredan las propiedades y el comportamiento de todas las clases a las que pertenecen en esa jerarquía.

- Recolección de basura

La recolección de basura (*garbage collection*) es la técnica por la cual el entorno de objetos se encarga de destruir automáticamente los objetos que hayan quedado sin ninguna referencia a ellos. Esto significa que el programador no debe preocuparse por la asignación o liberación de memoria, ya que el entorno la asignará al crear un nuevo objeto y la liberará cuando nadie lo esté usando.

3 El Lenguaje Unificado de Modelado

3.1 Concepto de UML

Los desarrolladores de software han utilizado lenguajes de modelado durante décadas para especificar, visualizar, construir, y documentar sistemas. El Lenguaje de Modelado Unificado (UML) es uno de esos lenguajes. UML permite a los miembros del equipo colaborar proporcionando un lenguaje común que se

puede aplicar a una multitud de sistemas diferentes. En esencia, permite comunicar soluciones en un lenguaje consistente, rico y expresivo.

Hoy en día UML se ha convertido en el estándar “de facto” para el modelado de sistemas de software. Aunque un desarrollador no escribiera nunca diagramas UML, necesariamente tendría que saber interpretar diagramas escritos por otros.

UML se ha expandido bastante desde su creación, y aunque tiene sus orígenes en el desarrollo de software, actualmente se aplica a muchos campos diferentes; puede comunicar cómo es la organización de una empresa, sus procesos de negocio, etc. UML pretende ser una forma común de capturar y expresar relaciones, comportamientos e ideas de alto nivel en una notación que es fácil de aprender y eficiente de escribir. UML es visual; aproximadamente todo en él tiene una representación gráfica.

Las raíces de UML se remontan a tres métodos diferentes: el Método Booch de Grady Booch, OMT de James Rumbaugh, y Objectory de Ivar Jacobson. Conocidos como “Los Tres Amigos”, Booch, Rumbaugh y Jacobson diseñaron lo que se convirtió en la primera versión de UML, en 1994. En 1997, UML fue aceptado por el Object Management Group (OMG) y lanzado como UML v1.1. Desde entonces UML ha madurado considerablemente; la revisión UML 2.0 que fue adoptada por el OMG en 2005.

Una de las principales motivaciones para el paso de UML 1.5 y UML 2.0 fue añadir la posibilidad de que los modeladores puedan capturar más comportamiento del sistema y aumentar la automatización. Una técnica relativamente nueva llamada Arquitectura Dirigida por el Modelo (*Model Driven Architecture* — MDA) ofrece el potencial de desarrollar modelos ejecutables que las herramientas pueden enlazar entre sí, elevando así el nivel de abstracción de los lenguajes de programación tradicionales. UML 2.0 es fundamental para el empleo de esta técnica.

Usos comunes de UML incluyen:

- Diseño de software
- Comunicación de procesos de negocio
- Capturar los requerimientos de un sistema
- Documentar un sistema, proceso u organización existente.

Hay algunas reglas de oro a tener en cuenta al utilizar UML:

- Casi todo en UML es opcional: Mostrar sólo lo que ayuda a aclarar el mensaje que se está tratando de transmitir, y dejar fuera lo que no es necesario.
- Raramente los modelos UML están completos: El truco está en no perder detalles importantes que podrían afectar el diseño del sistema. Saber lo que es un detalle clave viene dado por la experiencia; sin embargo, el uso de un proceso iterativo y las revisiones del modelo ayudan a profundizar en este aspecto.
- UML está diseñado para estar abierto a la interpretación: En los equipos de diseño es una buena práctica fijar conjuntamente un documento sobre directrices de modelado.
- UML está diseñado para que sea extensible: UML incluye varios mecanismos para permitir la personalización y el refinamiento del lenguaje.

3.2 Diagramas en UML

El bloque básico de construcción de UML es el diagrama. Hay varios tipos, algunos con propósitos muy específicos (p. e., los diagramas de tiempo) y algunos con usos más genéricos (p. e., los diagramas de clases).

Un modelo UML proporciona una vista de un sistema. Los usuarios nuevos a UML pueden caer en la trampa de tratar de modelar todo sobre su sistema con un solo diagrama y terminar perdiendo información crítica. O, en el otro extremo, pueden tratar de incorporar todos los posibles diagramas UML a su modelo,

complicando innecesariamente las cosas y convirtiendo el mantenimiento en una pesadilla. Ser hábil con UML significa entender lo que cada diagrama tiene que ofrecer y saber cuándo aplicarlo.

Al crear un modelo se debería tener presente al futuro destinatario del mismo. A un ingeniero de pruebas podría no interesarle la implementación de bajo nivel de un componente (un diagrama de secuencia), sino sólo la interfaz externa que ofrece (un diagrama de componentes). Hay que considerar pues quién va a utilizar cada diagrama.

UML 2.0 divide los diagramas en dos categorías:

- Diagramas estructurales: Se utilizan para capturar la organización física de las cosas en el sistema, esto es, cómo un objeto se relaciona con otro. Los diagramas estructurales son:
 - De clases: Utilizan clases e interfaces para capturar detalles acerca de las entidades que constituyen el sistema y las relaciones estáticas entre ellas.
 - De componentes: Muestran la organización y las dependencias existentes en la implementación de un sistema. Se pueden agrupar elementos más pequeños, como las clases, en componentes desplegables mayores.
 - De estructura compuesta: Muestran cómo los elementos en el sistema se combinan para realizar modelos complejos.
 - De despliegue: Muestra cómo el sistema se ejecuta y se asigna a varios elementos hardware.
 - De paquetes: Son tipos especiales de diagramas de clases. Usan la misma notación, pero su atención se centra en cómo se agrupan las clases e interfaces.
 - De objetos: Utilizan la misma sintaxis que los diagramas de clases y muestran cómo instancias reales de las clases están relacionadas a un instante específico de tiempo.
- Diagramas de comportamiento: Se centran en el comportamiento de los elementos en un sistema. Por ejemplo, se pueden utilizar diagramas de comportamiento para capturar requerimientos, operaciones y cambios de estado internos de elementos. Los diagramas de comportamiento son:
 - De actividad: Capturan el flujo de un comportamiento.
 - De comunicación: Se centra en los elementos que intervienen en un comportamiento en particular y en los mensajes que se intercambian.
 - De descripción de la interacción: Son versiones simplificadas de los diagramas de actividad; en lugar de hacer hincapié en cada paso de la actividad, enfatizan qué elementos están involucrados en la ejecución de esta actividad.
 - De secuencia: Describen el tipo y el orden de los mensajes transmitidos entre elementos durante la ejecución.
 - De máquina de estados: Capturan las transiciones del estado interno de un elemento.
 - De temporización: Enfatizan las especificaciones de temporización para los mensajes. A menudo se utilizan para modelar sistemas de tiempo real.
 - De casos de uso: Capturan los requerimientos funcionales de un sistema. Describen las interacciones entre el sistema y los usuarios, y entre el sistema y otros sistemas externos. Proporcionan una vista independiente de la implementación de lo que un sistema debe hacer y permiten al modelador centrarse en las necesidades del usuario en lugar de en los detalles de realización.

3.3 Utilización de UML en la metodología RUP

Es importante darse cuenta de que UML no es un proceso de software, aunque se ha diseñado para ser utilizado dentro de un proceso de software y tiene facetas claramente destinadas a ser parte de un enfoque de desarrollo iterativo, tal como el propuesto por la metodología RUP (también planteada por “Los Tres Amigos”).

De las cuatro fases del ciclo de vida RUP (comienzo, elaboración, construcción y transición), UML es de gran utilidad en las dos intermedias.

- En la fase de elaboración, los diagramas de casos de uso serán los motores de todo el proceso de desarrollo, por ser la herramienta de comunicación entre cliente y analista que permitirá plasmar los requerimientos.

Por otra parte, para establecer un modelo de dominio, es decir un modelo del mundo real al que la aplicación que se desea desarrollar ha de dar servicio, los diagramas de clases y los de actividades son de gran utilidad: los primeros sirven para establecer los conceptos de interés en el negocio y cómo se vinculan entre sí, y los segundos permiten describir el flujo de trabajo del negocio.

La fase de elaboración pretende llegar a definir la base arquitectónica, que se compondrá de la lista de casos de uso, del modelo de dominio y de la plataforma tecnológica. Esta última describirá las partes clave de la tecnología de implementación y cómo se acoplan; para ello resultarán de interés los diagramas de paquetes y los de despliegue.

- En la fase de construcción todos los diagramas UML pueden ser de aplicación. Por ejemplo, un diagrama de interacción es valioso para mostrar la manera en que interactuarán las clases para implementar un caso de uso, un diagrama de estado puede describir el ciclo de vida de una clase si éste resulta ser complicado, etc.

4 Bibliografía

- PILONE, D.; PITMAN, N. (2005): *UML 2.0 in a Nutshell*. O'Reilly
- FOWLER, M (1999): *UML Gota a Gota*. Addison Wesley. México
- Apuntes del módulo profesional “Entornos de Desarrollo” del I.E.S. Julio Verne.
- Wikipedia: “Orientación a Objetos”